

变量与数据类型

本章学习目标

- 理解变量的概念及其在 Shell 中的作用
 - 掌握变量的定义、赋值与引用方法
 - 熟悉变量命名规范和最佳实践
 - 理解环境变量与普通变量的区别
 - 掌握特殊变量的含义和用法
 - 学会使用 `readonly` 和 `unset` 管理变量
-

3.1 变量的基本概念

3.1.1 什么是变量？

变量就像是一个带标签的盒子，你可以把数据放进去，也可以随时取出来使用。标签就是变量名，盒子里的内容就是变量的值。

生活中的类比：

想象你有一个储物柜：

- 柜子上贴着标签"工具箱"（变量名）
- 柜子里放着锤子、螺丝刀（变量值）
- 你需要工具时，只要说"把工具箱拿来"，别人就知道去哪找

在 Shell 中也一样：

- `name="张三"` —— 创建一个叫 `name` 的盒子，放入"张三"
- `echo $name` —— 打开 `name` 盒子，取出里面的内容

3.1.2 Shell 变量的特点

与其他编程语言相比，Shell 变量有一些独特之处：

特点	说明	示例
弱类型	不需要声明类型，默认都是字符串	<code>age=25</code> （实际存储为字符串"25"）
动态创建	使用时自动创建，无需提前声明	直接 <code>name="Tom"</code> 即可
区分大小写	<code>Name</code> 和 <code>name</code> 是两个不同的变量	<code>Name="A"</code> <code>name="B"</code>
无需特殊符号定义	不像 PHP 的 <code>\$</code> ，定义时不加符号	<code>count=10</code> 而非 <code>\$count=10</code>
引用需加 <code>\$</code>	获取变量值时必须加 <code>\$</code>	<code>echo \$count</code>

3.2 变量的定义与使用

3.2.1 变量定义的基本语法

```
变量名=值
```

 最重要的规则：等号两边不能有空格！

【实操练习 3-1】变量定义的正确与错误示范

```
# ✅ 正确的定义方式
name="张三"
age=25
is_active=true

# ❌ 错误的定义方式（等号周围有空格）
name = "张三"      # 错误! Shell 会把 name 当成命令
age= 25            # 错误!
is_active =true    # 错误!
```

为什么不能有空格?

因为 Shell 把空格作为命令和参数的分隔符。`name = "张三"` 会被解释为：执行 `name` 命令，参数是 `=` 和 `"张三"`。

3.2.2 变量的引用

使用 `$变量名` 或 `${变量名}` 来获取变量的值：

```
name="Shell"

# 方式1：直接使用 $变量名
echo $name

# 方式2：使用 ${变量名}（推荐）
echo ${name}

# 方式3：在字符串中使用
echo "Hello, ${name}!"
```

什么时候必须用 `${}` ?

当变量名与其他字符紧邻时，必须使用花括号来明确边界：

```
file="report"

# 想输出 "report_backup"
echo $file_backup      # 错误! Shell 会找 $file_backup 这个变量
echo ${file}_backup    # 正确! 输出 report_backup
```

【图3-1：变量引用的两种方式】

变量引用方式对比	
方式一：\$变量名	方式二：\${变量名}
<pre>name="Shell" echo \$name</pre> ↓ Shell	<pre>name="Shell" echo \${name}</pre> ↓ Shell
⚠️ 简单场景可用 ⚠️ 边界不清时会出错	✅ 推荐使用 ✅ 边界清晰不易错
示例： <pre>file="log" echo "\$file_2024" → 空 (找不到变量 file_2024) echo "\${file}_2024" → log_2024 ✓</pre>	

3.2.3 变量赋值的多种方式

```
# 1. 直接赋值
name="张三"

# 2. 将命令的输出赋值给变量
current_date=$(date +%Y-%m-%d)    # 推荐方式
current_time=`date +%H:%M:%S`     # 旧式写法, 不推荐
```

```
# 3. 从另一个变量赋值
new_name=$name

# 4. 拼接字符串赋值
greeting="Hello, ${name}!"

# 5. 读取用户输入赋值
read -p "请输入你的名字: " username
```

【实操练习 3-2】变量赋值的多种方式

```
#!/bin/bash
# 文件名: var_assign.sh

# 直接赋值
os_name="CentOS"
echo "操作系统: $os_name"

# 命令输出赋值
kernel_version=$(uname -r)
echo "内核版本: $kernel_version"

# 变量拼接
full_info="${os_name} - ${kernel_version}"
echo "完整信息: $full_info"

# 用户输入赋值
read -p "请输入你的名字: " user_name
echo "你好, ${user_name}! "
```

3.3 变量命名规范

3.3.1 命名规则（必须遵守）

规则	正确示例	错误示例	说明
只能包含字母、数字、下划线	<code>user_name</code>	<code>user-name</code>	不能用减号、点号等
不能以数字开头	<code>file1</code>	<code>1file</code>	可以包含数字，但不能开头
不能使用关键字	<code>my_if</code>	<code>if</code>	不能用 Shell 的保留字
区分大小写	<code>Name</code> ≠ <code>name</code>	—	大小写是不同变量

Shell 保留关键字（不能用作变量名）：

```
if, then, else, elif, fi, case, esac, for, while, until, do, done,
in, function, select, time, until, break, continue, return, exit
```

3.3.2 命名惯例（推荐遵守）

惯例	示例	说明
普通变量用小写	<code>file_name</code> , <code>count</code>	区别于环境变量
环境变量用大写	<code>PATH</code> , <code>HOME</code> , <code>MY_APP_HOME</code>	沿用 Unix 传统
多单词用下划线连接	<code>backup_dir</code> , <code>max_retry</code>	蛇形命名法
常量用大写	<code>readonly MAX_SIZE=100</code>	表示不可修改
名称要有意义	<code>user_count</code>	避免 <code>a</code> , <code>x</code> , <code>tmp1</code>

【好的命名示例】

```
# ✅ 好的命名
backup_dir="/data/backup"
max_retry_count=3
is_debug_mode=true
db_connection_string="mysql://localhost:3306"

# ❌ 不好的命名
a="/data/backup"           # 太简短，不知道是什么
dir1="/data/backup"        # 数字后缀不表意
backupDirPath="/data"      # 驼峰命名不符合Shell惯例
```

3.4 引号的使用

在 Shell 中，引号的使用非常重要，不同的引号有不同的作用。

3.4.1 三种引号对比

引号类型	符号	变量替换	命令替换	特殊字符	典型用途
双引号	"..."	✅ 替换	✅ 替换	部分保留	包含变量的字符串
单引号	'...'	❌ 不替换	❌ 不替换	全部保留	原样输出的字符串
反引号	...	—	执行命令	—	命令替换（不推荐）

【实操练习 3-3】引号的区别演示

```
#!/bin/bash
# 文件名: quote_demo.sh

name="Shell"
current_date=$(date +%Y-%m-%d)

echo "==== 双引号测试 ====="
echo "Hello, $name"                # 变量被替换
echo "Today is $(date +%Y-%m-%d)" # 命令被执行
echo "Path is: $PATH"             # 变量被替换
```

```

echo ""
echo "==== 单引号测试 ====="
echo 'Hello, $name' # 原样输出 $name
echo 'Today is $(date +%Y-%m-%d)' # 原样输出
echo 'Path is: $PATH' # 原样输出

echo ""
echo "==== 不加引号测试 ====="
echo Hello, $name # 变量被替换，但可能有空格问题

```

执行结果：

```

==== 双引号测试 =====
Hello, Shell
Today is 2024-01-01
Path is: /usr/local/bin:/usr/bin:/bin

==== 单引号测试 =====
Hello, $name
Today is $(date +%Y-%m-%d)
Path is: $PATH

==== 不加引号测试 =====
Hello, Shell

```

3.4.2 什么时候用什么引号？

```

# 1. 需要变量替换 → 用双引号
message="Hello, ${username}!"

# 2. 原样输出，不要任何替换 → 用单引号
echo 'Use $HOME to get home directory'

# 3. 字符串中包含单引号 → 用双引号
echo "It's a good day"

# 4. 字符串中包含双引号 → 用单引号或转义
echo 'He said "Hello"'

```

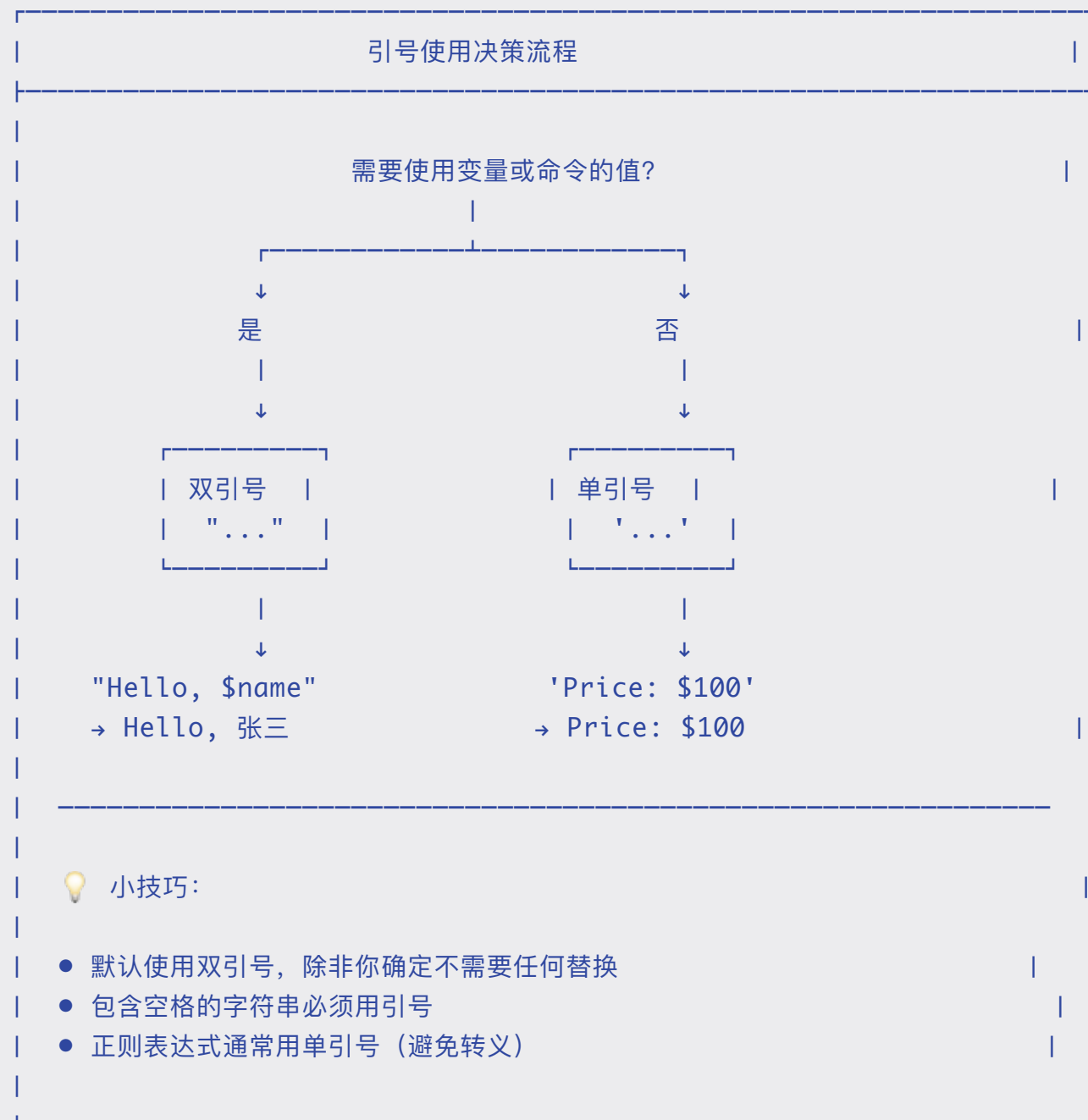
```
echo "He said \"Hello\""
```

5. 包含空格的值 → 必须用引号

```
file_path="/path/to/my file.txt"
```

双引号保留空格

【图3-2：引号使用决策流程】



3.5 环境变量

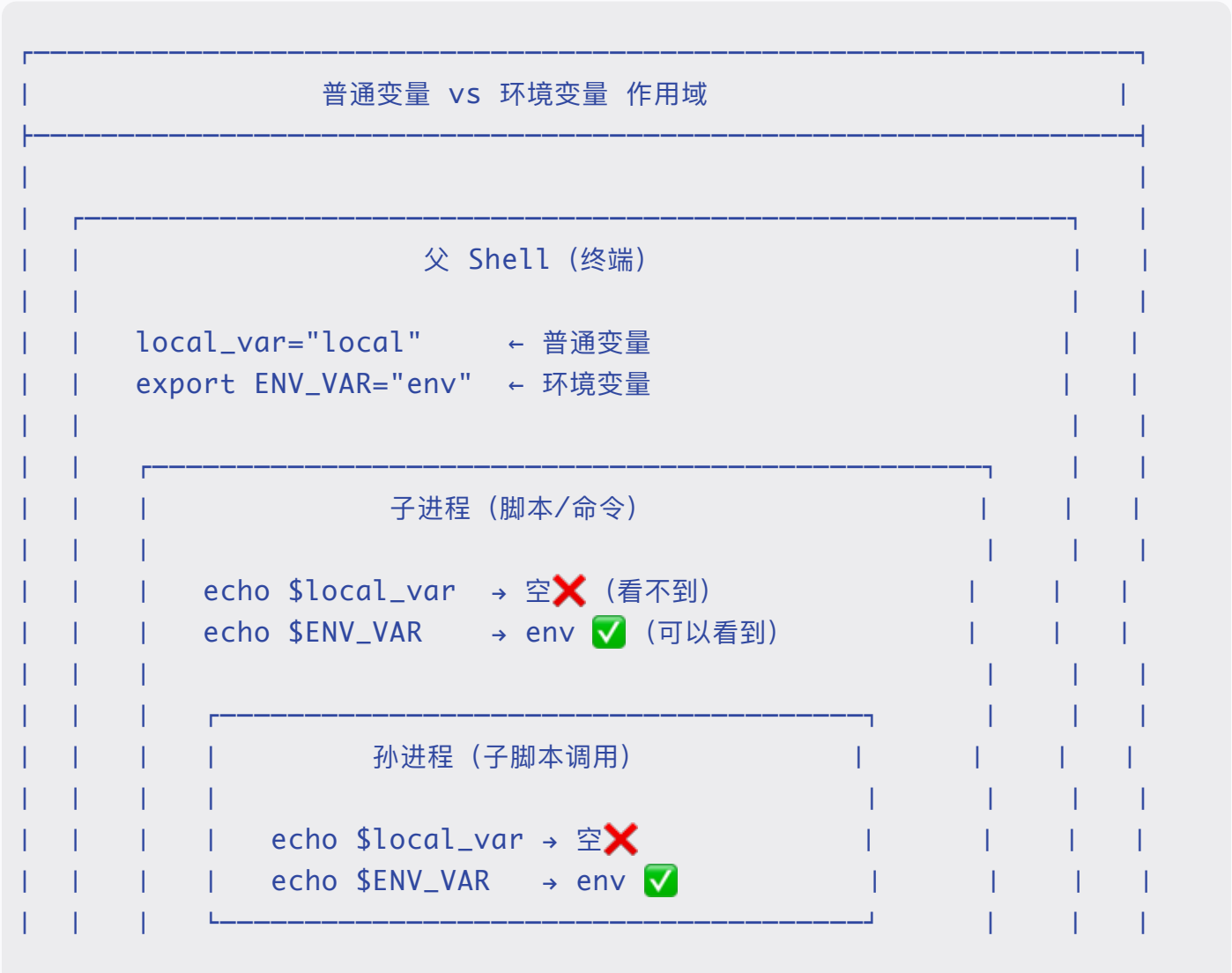
3.5.1 什么是环境变量？

环境变量是系统级别的变量，可以被当前 Shell 及其所有子进程访问。

普通变量 vs 环境变量：

对比项	普通变量	环境变量
作用范围	仅当前 Shell	当前 Shell 及所有子进程
定义方式	<code>var=value</code>	<code>export VAR=value</code>
命名惯例	小写	大写
生命周期	当前 Shell 退出即消失	可配置为永久有效

【图3-3：普通变量与环境变量的作用域】





环境变量会"向下传递"给所有子进程，普通变量不会

3.5.2 常用的系统环境变量

变量名	说明	示例值
HOME	当前用户的家目录	/root 或 /home/username
PATH	可执行文件搜索路径	/usr/local/bin:/usr/bin:/bin
USER	当前登录用户名	root
SHELL	当前使用的 Shell	/bin/bash
PWD	当前工作目录	/root/scripts
LANG	系统语言设置	en_US.UTF-8
HOSTNAME	主机名	localhost
PS1	命令提示符格式	[\u@\h \W]\\$

【实操练习 3-4】查看系统环境变量

```
# 查看所有环境变量
env

# 查看特定环境变量
echo $HOME
echo $PATH
echo $USER

# 查看所有变量（包括普通变量）
set

# 只看环境变量（另一种方式）
export -p
```

3.5.3 定义和导出环境变量

```
# 方式1：分两步，先定义后导出
MY_VAR="hello"
export MY_VAR

# 方式2：一步完成（推荐）
export MY_VAR="hello"

# 查看是否成功导出
echo $MY_VAR
```

【实操练习 3-5】环境变量与普通变量的区别演示

```
# 1. 创建测试脚本
cat > ~/scripts/check_var.sh << 'EOF'
#!/bin/bash
echo "在子进程中："
echo "  LOCAL_VAR = $LOCAL_VAR"
echo "  EXPORT_VAR = $EXPORT_VAR"
EOF
chmod +x ~/scripts/check_var.sh

# 2. 在当前 Shell 中定义变量
```

```
LOCAL_VAR="I am local"
export EXPORT_VAR="I am exported"

# 3. 在当前 Shell 中查看
echo "在当前 Shell 中: "
echo "  LOCAL_VAR = $LOCAL_VAR"
echo "  EXPORT_VAR = $EXPORT_VAR"

# 4. 运行子进程脚本
echo ""
~/scripts/check_var.sh
```

执行结果：

```
在当前 Shell 中：
LOCAL_VAR = I am local
EXPORT_VAR = I am exported

在子进程中：
LOCAL_VAR =                ← 空！普通变量不传递给子进程
EXPORT_VAR = I am exported  ← 环境变量成功传递
```

3.5.4 永久设置环境变量

在终端中设置的环境变量只在当前会话有效，关闭终端就没了。要永久生效，需要写入配置文件：

配置文件	作用范围	说明
<code>/etc/profile</code>	所有用户	系统级配置，需要 root 权限
<code>/etc/bashrc</code>	所有用户	系统级 Bash 配置
<code>~/.bash_profile</code>	当前用户	用户登录时执行
<code>~/.bashrc</code>	当前用户	每次打开新终端时执行（推荐）

【实操练习 3-6】永久设置环境变量

```
# 1. 编辑用户的 bashrc 文件
vim ~/.bashrc

# 2. 在文件末尾添加
export MY_APP_HOME="/opt/myapp"
export PATH="$PATH:$MY_APP_HOME/bin"

# 3. 保存退出后，使配置生效
source ~/.bashrc

# 4. 验证
echo $MY_APP_HOME
echo $PATH
```

3.6 特殊变量

Shell 预定义了一些特殊变量，在脚本中非常有用。

3.6.1 位置参数变量

位置参数变量用于接收脚本或函数的参数：

变量	说明	示例
<code>\$0</code>	脚本文件名	<code>./test.sh</code>
<code>\$1 ~ \$9</code>	第1到第9个参数	第一个参数、第二个参数...
<code>\${10}</code>	第10个及以后的参数	必须用花括号
<code>\$#</code>	参数的个数	如果传了3个参数，值为3
<code>\$*</code>	所有参数（作为一个整体）	<code>"\$*" → "arg1 arg2 arg3"</code>
<code>@</code>	所有参数（作为独立个体）	<code>"@" → "arg1" "arg2" "arg3"</code>

【实操练习 3-7】位置参数变量演示

```
# 创建脚本
cat > ~/scripts/params.sh << 'EOF'
#!/bin/bash
echo "脚本名称: $0"
echo "第一个参数: $1"
echo "第二个参数: $2"
echo "第三个参数: $3"
echo "参数个数: $# "
echo "所有参数(\$*): $*"
echo "所有参数(\$@): @$"
EOF
chmod +x ~/scripts/params.sh

# 执行脚本, 传入参数
~/scripts/params.sh apple banana cherry
```

执行结果:

```
脚本名称: /root/scripts/params.sh
第一个参数: apple
第二个参数: banana
第三个参数: cherry
参数个数: 3
所有参数($*): apple banana cherry
所有参数($@): apple banana cherry
```

3.6.2 *和@ 的区别

这两个看起来一样, 但在双引号中行为不同:

```
# 创建演示脚本
cat > ~/scripts/star_vs_at.sh << 'EOF'
#!/bin/bash

echo "=== 使用 \$* ==="
for arg in "$*"; do
    echo "参数: $arg"
done
```

```
echo ""
echo "=== 使用 \$@ ==="
for arg in "$@"; do
    echo "参数: $arg"
done
EOF
chmod +x ~/scripts/star_vs_at.sh

# 执行
~/scripts/star_vs_at.sh "hello world" foo bar
```

执行结果：

```
=== 使用 $* ===
参数: hello world foo bar      ← 所有参数合并成一个字符串

=== 使用 @$ ===
参数: hello world             ← 第一个参数（包含空格，保持原样）
参数: foo                     ← 第二个参数
参数: bar                     ← 第三个参数
```

 **最佳实践：** 在 for 循环中遍历参数时，推荐使用 `"$@"`，它能正确处理包含空格的参数。

3.6.3 状态变量

变量	说明	典型值
<code>\$?</code>	上一条命令的退出状态码	0 表示成功，非0表示失败
<code>\$\$</code>	当前脚本的进程ID (PID)	如 <code>12345</code>
<code>\$_</code>	最后一个后台进程的 PID	如 <code>12346</code>
<code>\$_</code>	上一条命令的最后一个参数	—

【实操练习 3-8】状态变量演示

```
#!/bin/bash
# 文件名: status_var.sh

echo "当前脚本的 PID: $$"

# 执行一个成功的命令
ls /tmp > /dev/null
echo "ls 命令执行后 \ $? = ?"

# 执行一个失败的命令
ls /nonexistent_dir 2> /dev/null
echo "访问不存在的目录后 \ $? = ?"

# 后台运行一个命令
sleep 10 &
echo "后台进程的 PID: $!"
```

执行结果：

```
当前脚本的 PID: 23456
ls 命令执行后 $? = 0
访问不存在的目录后 $? = 2
后台进程的 PID: 23457
```

3.6.4 \$? 的重要性

`$?` 是脚本编程中最常用的特殊变量之一，用于判断命令是否执行成功：

```
#!/bin/bash

# 尝试创建目录
mkdir -p /tmp/mydir

# 检查是否成功
if [ $? -eq 0 ]; then
    echo "目录创建成功"
else
    echo "目录创建失败"
```

```
fi

# 更简洁的写法
if mkdir -p /tmp/mydir2; then
    echo "目录创建成功"
else
    echo "目录创建失败"
fi
```

常见命令的退出状态码：

状态码	含义	示例场景
0	成功	命令正常执行完成
1	一般错误	命令执行出错
2	误用 Shell 命令	参数错误、语法错误
126	命令不可执行	没有执行权限
127	命令未找到	命令不存在
128+N	被信号 N 终止	如 130 表示被 Ctrl+C 终止

3.7 变量的高级操作

3.7.1 只读变量 (readonly)

使用 `readonly` 可以将变量设为只读，防止被修改或删除：

```
# 定义只读变量
readonly PI=3.14159
readonly COMPANY="Acme Inc."

# 尝试修改（会报错）
PI=3.14
# 输出: -bash: PI: readonly variable

# 尝试删除（也会报错）
unset PI
# 输出: -bash: unset: PI: cannot unset: readonly variable
```

使用场景：

- 定义程序中不应该被修改的常量
- 保护关键配置不被意外覆盖

```
#!/bin/bash
# 配置文件常量（不允许脚本其他部分修改）
readonly CONFIG_FILE="/etc/myapp/config.conf"
readonly LOG_DIR="/var/log/myapp"
readonly MAX_CONNECTIONS=100
```

3.7.2 删除变量 (unset)

使用 `unset` 可以删除变量，释放内存：

```

# 定义变量
name="Shell"
echo $name          # 输出: Shell

# 删除变量
unset name
echo $name          # 输出: 空

# 验证变量是否存在
if [ -z "$name" ]; then
    echo "变量 name 不存在或为空"
fi

```

 **注意：** `unset` 不能删除只读变量。

3.7.3 变量默认值

当变量可能为空时，可以使用默认值语法：

语法	说明	示例
<code>\${var:-default}</code>	如果 var 为空或未定义，返回 default	<code>\${name:-Guest}</code>
<code>\${var:=default}</code>	如果 var 为空或未定义，设置 var=default 并返回	<code>\${name:=Guest}</code>
<code>\${var:+value}</code>	如果 var 已定义且不为空，返回 value	<code>\${name:+has value}</code>
<code>\${var:?message}</code>	如果 var 为空或未定义，打印错误信息并退出	<code>\${name:?变量未定 义}</code>

【实操练习 3-9】变量默认值用法

```

#!/bin/bash
# 文件名: default_value.sh

```

```

# 场景1：使用默认值，但不修改变量
unset username
echo "欢迎, ${username:-访客}!"      # 输出：欢迎，访客！
echo "username = $username"          # 输出：username = （仍为空）

# 场景2：设置默认值并保存
unset server_port
: ${server_port:=8080}                # 如果未设置，则设为8080
echo "端口：$server_port"            # 输出：端口：8080

# 场景3：检查变量是否已设置
db_host="localhost"
echo "状态：${db_host:+已配置}"      # 输出：状态：已配置

# 场景4：变量未定义时报错退出
# ${REQUIRED_VAR:?这是必需的变量}    # 如果未定义会报错并退出脚本

```

实用示例：为脚本参数提供默认值

```

#!/bin/bash
# 如果没有传参数，使用默认值

# 方式1：检查后赋值
if [ -z "$1" ]; then
    filename="default.txt"
else
    filename="$1"
fi

# 方式2：使用默认值语法（更简洁）
filename=${1:-default.txt}
count=${2:-10}

echo "处理文件：$filename，数量：$count"

```

3.8 字符串操作

虽然 Shell 中所有变量本质上都是字符串，但提供了一些方便的字符串操作功能。

3.8.1 字符串长度

```
str="Hello, Shell!"  
echo ${#str}      # 输出: 13
```

3.8.2 字符串截取

```
str="Hello, Shell World!"  
  
# 从位置5开始截取到末尾（位置从0开始）  
echo ${str:5}      # 输出: , Shell World!  
  
# 从位置7开始截取5个字符  
echo ${str:7:5}     # 输出: Shell  
  
# 从右边开始截取（注意冒号前的空格）  
echo ${str: -6}     # 输出: World!  
echo ${str: -6:5}   # 输出: World
```

3.8.3 字符串替换

```
path="/usr/local/bin:/usr/bin:/bin"  
  
# 替换第一个匹配  
echo ${path/bin/BIN}      # 输出: /usr/local/BIN:/usr/bin:/bin  
  
# 替换所有匹配  
echo ${path//bin/BIN}     # 输出: /usr/local/BIN:/usr/BIN:/BIN  
  
# 从开头匹配替换  
str="hello_hello_world"  
echo ${str/#hello/HELLO}  # 输出: HELLO_hello_world  
  
# 从结尾匹配替换  
echo ${str/%world/WORLD}  # 输出: hello_hello_WORLD
```

3.8.4 字符串删除

```
filename="/path/to/file.txt.bak"

# 从左边删除最短匹配
echo ${filename#*/}          # 输出: path/to/file.txt.bak

# 从左边删除最长匹配
echo ${filename##*/}        # 输出: file.txt.bak (常用于获取文件名)

# 从右边删除最短匹配
echo ${filename%.*}         # 输出: /path/to/file.txt

# 从右边删除最长匹配
echo ${filename%%.*}        # 输出: /path/to/file (删除所有扩展名)
```

【实操练习 3-10】字符串操作综合示例

```
#!/bin/bash
# 文件名: string_ops.sh

filepath="/home/user/documents/report_2024.txt"

echo "原始路径: $filepath"
echo "字符串长度: ${#filepath}"

# 提取文件名 (删除路径)
filename=${filepath##*/}
echo "文件名: $filename"

# 提取目录路径 (删除文件名)
dirname=${filepath%/*}
echo "目录路径: $dirname"

# 提取文件扩展名
extension=${filename##*.}
echo "扩展名: $extension"

# 提取不带扩展名的文件名
basename=${filename%.*}
```

```
echo "基本名: $basename"

# 替换年份
new_filename=${filename/2024/2025}
echo "新文件名: $new_filename"
```

执行结果：

```
原始路径: /home/user/documents/report_2024.txt
字符串长度: 39
文件名: report_2024.txt
目录路径: /home/user/documents
扩展名: txt
基本名: report_2024
新文件名: report_2025.txt
```

3.9 综合实战：配置读取脚本

让我们编写一个实用的脚本，演示本章学到的变量操作技巧。

【实操练习 3-11】配置文件读取脚本

```
#!/bin/bash
#####
# 文件名: config_reader.sh
# 功能描述: 读取配置文件并显示配置信息
# 使用方法: ./config_reader.sh [配置文件路径]
#####

#-----
# 常量定义
#-----
readonly SCRIPT_NAME=${0##*/}
readonly DEFAULT_CONFIG="./app.conf"

#-----
# 默认配置值
#-----
```

```
DB_HOST=${DB_HOST:-localhost}
DB_PORT=${DB_PORT:-3306}
DB_USER=${DB_USER:-root}
APP_NAME=${APP_NAME:-MyApp}
LOG_LEVEL=${LOG_LEVEL:-INFO}

#-----
# 显示帮助信息
#-----
show_help() {
    echo "用法: $SCRIPT_NAME [选项] [配置文件]"
    echo ""
    echo "选项:"
    echo "  -h, --help      显示帮助信息"
    echo ""
    echo "如果不指定配置文件, 将使用默认值"
}

#-----
# 解析配置文件
#-----
parse_config() {
    local config_file="$1"

    if [ -f "$config_file" ]; then
        echo "正在读取配置文件: $config_file"
        echo ""

        # 逐行读取配置文件
        while IFS='=' read -r key value; do
            # 跳过空行和注释
            [[ -z "$key" || "$key" =~ ^# ]] && continue

            # 去除首尾空格
            key=$(echo "$key" | xargs)
            value=$(echo "$value" | xargs)

            # 根据键名设置变量
            case "$key" in
                DB_HOST)    DB_HOST="$value" ;;
                DB_PORT)    DB_PORT="$value" ;;
                DB_USER)    DB_USER="$value" ;;
```

```

        APP_NAME)    APP_NAME="$value" ;;
        LOG_LEVEL)   LOG_LEVEL="$value" ;;
    esac
done < "$config_file"
else
    echo "配置文件不存在: $config_file"
    echo "使用默认配置值"
    echo ""
fi
}

#-----
# 显示配置信息
#-----
show_config() {
    echo "===== "
    echo "          当前配置信息"
    echo "===== "
    echo ""
    echo "【应用配置】 "
    echo "  应用名称: $APP_NAME"
    echo "  日志级别: $LOG_LEVEL"
    echo ""
    echo "【数据库配置】 "
    echo "  主机地址: $DB_HOST"
    echo "  端口号:   $DB_PORT"
    echo "  用户名:   $DB_USER"
    echo ""
    echo "===== "
}

#-----
# 主程序
#-----
main() {
    # 检查参数
    case "${1:-}" in
        -h|--help)
            show_help
            exit 0
            ;;
    esac
}

```

```
# 配置文件路径（使用传入参数或默认值）
local config_file=${1:-$DEFAULT_CONFIG}

# 显示脚本信息
echo ""
echo "脚本名称: $SCRIPT_NAME"
echo "进程 PID: $$"
echo "参数个数: $# "
echo ""

# 解析配置并显示
parse_config "$config_file"
show_config
}

# 运行主程序
main "$@"
```

创建测试配置文件：

```
cat > ~/scripts/app.conf << 'EOF'
# 应用配置文件
# 作者：学员

# 应用设置
APP_NAME=ProductionApp
LOG_LEVEL=DEBUG

# 数据库设置
DB_HOST=192.168.1.100
DB_PORT=3307
DB_USER=admin
EOF
```

运行测试：

```
cd ~/scripts
chmod +x config_reader.sh

# 使用默认配置
./config_reader.sh

# 使用指定配置文件
./config_reader.sh app.conf

# 查看帮助
./config_reader.sh --help
```

3.10 本章小结

本章我们学习了以下核心内容：

变量基础：

1. 变量定义时等号两边不能有空格
2. 引用变量使用 `$变量名` 或 `${变量名}`
3. 变量命名要有意义，遵循规范

引号使用：

1. 双引号：允许变量替换，保留空格
2. 单引号：完全原样输出，不做任何替换
3. 包含空格的值必须用引号

环境变量：

1. 普通变量只在当前 Shell 有效
2. `export` 导出的环境变量可传递给子进程
3. 永久环境变量需写入配置文件（如 `~/.bashrc`）

特殊变量：

1. `$0` ~ `$9`、`$#`、`$@`、`$*`：位置参数相关

2. `$?`：上条命令的退出状态
3. `$$`：当前进程 PID

高级操作：

1. `readonly`：定义只读常量
 2. `unset`：删除变量
 3. `${var:-default}`：变量默认值
 4. 字符串截取、替换、删除操作
-

3.11 课后练习

练习题 1（基础）：编写脚本，定义变量存储你的姓名、年龄、城市，然后用一句话输出这些信息。

练习题 2（基础）：解释以下代码的输出结果：

```
name="Shell"
echo '$name'
echo "$name"
echo "${name}_script"
echo "$name_script"
```

练习题 3（进阶）：编写脚本 `greet.sh`，接收一个参数作为用户名：

- 如果提供了参数，输出 "Hello, 用户名!"
- 如果没有提供参数，输出 "Hello, Guest!"

练习题 4（进阶）：编写脚本，接收一个文件路径作为参数，输出：

- 文件的完整路径
- 文件的目录部分
- 文件名（不含路径）
- 文件扩展名
- 不含扩展名的文件名

练习题 5（综合）：编写一个脚本 `env_setup.sh`，要求：

- 定义只读常量 `APP_VERSION="1.0.0"`
- 设置环境变量 `MY_APP_HOME` 为 `/opt/myapp`
- 设置 `PATH` 包含 `$MY_APP_HOME/bin`
- 输出所有设置的值
- 创建一个子脚本来验证环境变量是否传递成功